

UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería



Proyecto Redes

Repositorio: <https://github.com/GenserDev/Redes-Proyecto-1>

Servidor MCP remoto: <https://logistics-mcp.mcp-chatbot.workers.dev/health>

Genser Catalán - 23401

1. Introducción

Este proyecto consiste en un chatbot de terminal que funciona como anfitrión de Model Context Protocol (MCP). El chatbot se comunica con un modelo de lenguaje a través de su API, mantiene el contexto de la conversación y se conecta a varios servidores MCP que le dan acceso a herramientas externas: el sistema de archivos, un repositorio Git y un servidor propio de logística que se ejecuta tanto de forma local como en la nube.

MCP utiliza JSON-RPC 2.0, un protocolo de la capa de aplicación. Siguiendo lo indicado en el enunciado, todo el protocolo se implementó de forma manual, no se utilizó ningún SDK ni framework de MCP. El código construye, delimita, envía, recibe y valida cada mensaje por su cuenta, tanto del lado del cliente como del lado del servidor.

Funcionalidades implementadas

#	Funcionalidad	Estado
1	Conexión con un LLM a nivel de su API	Implementada
2	Mantener contexto en una sesión	Implementada
3	Log de las interacciones con los servidores MCP	Implementada
4	Uso de los servidores MCP oficiales Filesystem y Git	Implementada
5	Servidor MCP propio ejecutado localmente	Implementada
6	El mismo servidor MCP ejecutado de forma remota	Implementada
7	Análisis de la comunicación con Wireshark	Implementada
8, 9, 10	Reporte escrito	Implementada

Tecnologías utilizadas

Componente	Tecnología
Lenguaje y entorno	Node.js 24, JavaScript con módulos ESM
Dependencias	chalk y dotenv
Protocolo	JSON-RPC 2.0, revisión MCP 2025-06-18
Modelo de lenguaje	Google Gemini 3.7 Flash (también Groq y Anthropic)
Servidores MCP	Filesystem (Node) y Git (Python)
Nube	Cloudflare Workers, desplegado con Wrangler
Análisis de red	Wireshark y tshark

El chatbot se conecta a cuatro servidores MCP. Los dos primeros son oficiales de Anthropic y se ejecutan como procesos hijos. El tercero y el cuarto son el mismo servidor propio, alcanzado por dos vías distintas.

Servidor	Transporte	Herramientas
filesystem	stdio	14
git	stdio	12
logistics	stdio	4
logistics-remote	HTTP	4

2. Especificación del servidor MCP desarrollado

El caso de uso elegido, aprobado previamente por el catedrático, es el de una empresa guatemalteca de paquetería. El servidor permite que un chatbot de atención al cliente cotice un envío, lo registre, lo rastree y liste los envíos de un cliente.

La lógica del dominio y el ruteo de métodos no saben nada del transporte. Gracias a eso, el mismo código es utilizado por el servidor local sobre stdio y por el servidor remoto sobre HTTP, sin duplicarse.

Identidad y capacidades

Campo	Valor
Nombre del servidor	logistics-mcp
Versión	1.0.0
Revisión del protocolo	2025-06-18
Capacidades	Únicamente tools (sin resources, prompts ni sampling)

Métodos implementados

Método	Tipo	Parámetros	Resultado
initialize	Solicitud	protocolVersion, capabilities, clientInfo	protocolVersion, capabilities, serverInfo
notifications/initialized	Notificación	ninguno	no lleva respuesta
ping	Solicitud	ninguno	objeto vacío
tools/list	Solicitud	ninguno	arreglo de herramientas
tools/call	Solicitud	name, arguments	content, isError

Endpoints

Transporte	Dirección	Delimitación de mensajes
stdio	node servers/logistics/stdio-server.js	Un mensaje JSON por línea en stdin y stdout
HTTP	POST https://logistics-mcp.mcp-chatbot.workers.dev/mcp	Un mensaje JSON por cuerpo de solicitud
HTTP	GET https://logistics-mcp.mcp-chatbot.workers.dev/health	Verificación de estado, fuera de MCP

Herramientas

Herramienta	Parámetros obligatorios	Opcionales	Devuelve
quote_shipment	origin, destination, weight_kg	service_level	Precio en quetzales, días de tránsito y fecha estimada
create_shipment	customer, origin, destination, weight_kg	service_level	Número de guía, precio y sucursal de entrega
track_shipment	tracking_number	—	Estado actual e historial completo de eventos
list_shipments	customer	status	Envíos que coinciden con el filtro

Las reglas del dominio son tres zonas de cobertura (metro, central y remota) que determinan la tarifa base, el costo por kilogramo y el tiempo de tránsito, tres niveles de servicio que multiplican el precio y acortan el tiempo, un límite de 70 kilogramos por paquete, y fechas de entrega calculadas en días hábiles.

Manejo de errores

El servidor distingue dos tipos de falla, y esa distinción es visible en las capturas de red. Un error de protocolo significa que el mensaje en sí estuvo mal formado, un error de dominio significa que el mensaje fue correcto y la respuesta es negativa.

Situación	Se reporta como
Método inexistente	Error JSON-RPC, código -32601
Cuerpo que no es JSON válido	Error JSON-RPC, código -32700, con id nulo
tools/call sin el parámetro name	Error JSON-RPC, código -32602
Número de guía inexistente	Respuesta exitosa con isError en verdadero
Ciudad sin cobertura o peso mayor a 70 kg	Respuesta exitosa con isError en verdadero

3. Análisis de la comunicación con Wireshark

Se tomaron dos capturas de la misma sesión MCP contra el mismo servidor. La primera es contra el servidor desplegado en Cloudflare sobre HTTPS, que es el escenario que pide el enunciado. La segunda es contra ese mismo servidor ejecutado localmente sin cifrado, lo que permite leer el contenido directamente y compararlo. Ambas fueron generadas por el propio cliente del proyecto.

Captura	Servidor	Transporte	Paquetes
mcp-remote-tls.pcapng	Cloudflare Workers	TCP/443, TLS 1.3, HTTP/1.1	350
mcp-local-plaintext.pcapng	127.0.0.1:8787	TCP/8787, HTTP/1.1 sin cifrar	76

Clasificación de los mensajes JSON-RPC

JSON-RPC 2.0 define tres formas de mensaje, y cada una se reconoce por lo que contiene el objeto JSON. Una solicitud lleva method e id, y espera respuesta. Una notificación lleva method pero no lleva id, y no recibe respuesta. Una respuesta lleva id junto con result o error, y se asocia a su solicitud por medio de ese id.

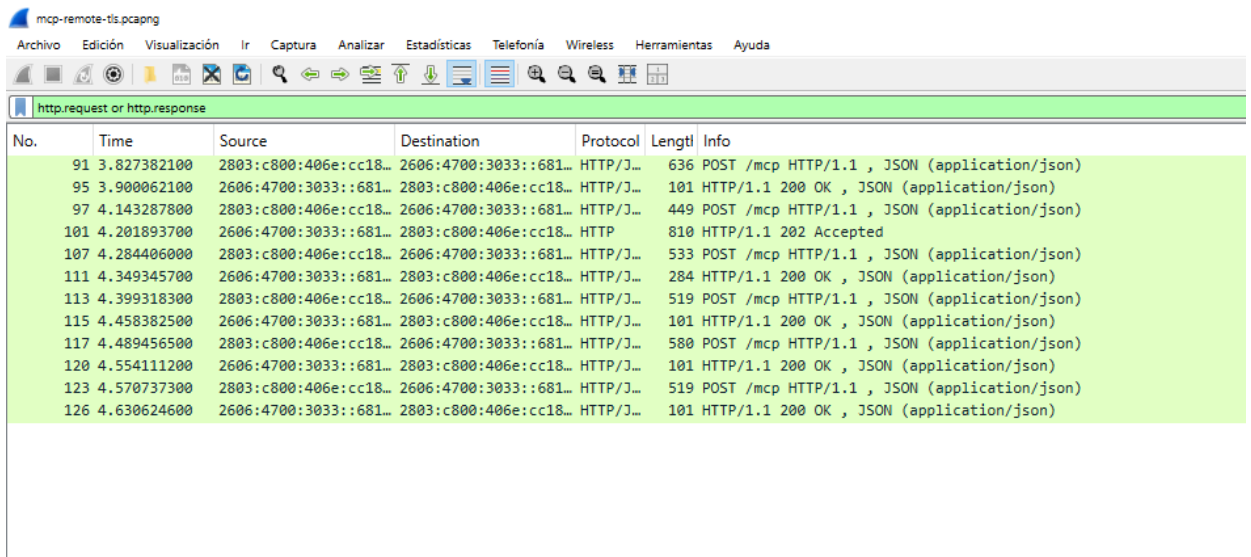


Figura 1. Los doce mensajes MCP descifrados de la sesión contra Cloudflare

Cuáles son los mensajes de sincronización

Los tres primeros mensajes forman el enlace inicial del protocolo. En la trama 91 el cliente anuncia la revisión que habla y sus capacidades; en la trama 95 el servidor responde con la revisión que usará y las capacidades que tiene, que en nuestro caso son únicamente tools.

La trama 97 es la notificación `notifications/initialized`, con la que el cliente confirma que el enlace quedó establecido, y es el único mensaje de toda la sesión sin id. El servidor responde con HTTP 202 y un cuerpo vacío en la trama 101: en la capa HTTP sí regresó algo, pero en la capa JSON-RPC no regresó nada, porque una notificación no tiene respuesta. Algo similar ocurre en la trama 126, donde una guía inexistente viaja como HTTP 200 con `isError` dentro del resultado, y no como un error de protocolo.

Capa de enlace

La captura remota se tomó sobre la interfaz Wi-Fi y las tramas son Ethernet II. El campo EtherType vale 0x86dd, que indica que el contenido es IPv6. Las tramas más grandes miden 1434 bytes, justo por debajo de la MTU de 1500 bytes.

La dirección MAC de destino es la del router, no la del servidor. Las direcciones MAC solo tienen sentido dentro del segmento local: cada trama va dirigida a la puerta de enlace, que la descarta y construye una nueva para el siguiente salto. La identidad del servidor vive una capa más arriba.

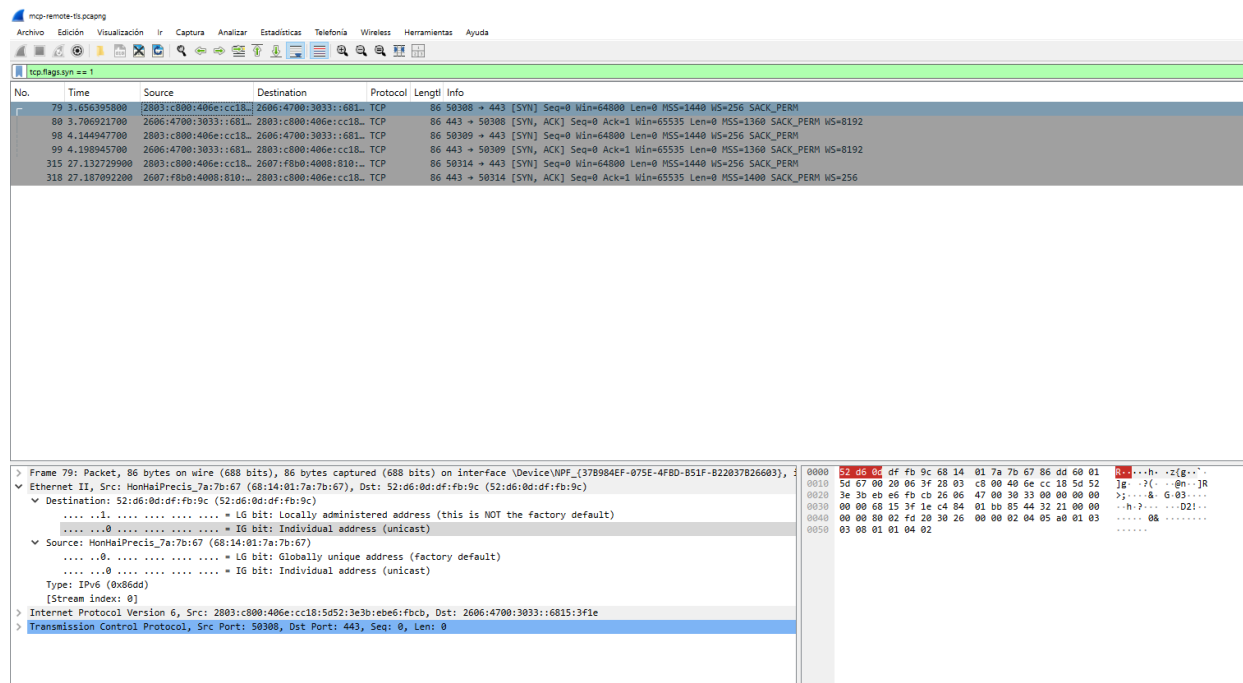


Figura 2. Saludo de tres vías de TCP y detalle de las capas de enlace y red

Capa de red

El tráfico viaja sobre IPv6, desde 2803:c800:406e:cc18:5d52:3e3b:ebe6:fbcb hacia 2606:4700:3033::6815:3f1e, dentro del rango de Cloudflare. El límite de saltos vale 63 al salir y 55 al regresar, lo que indica que las respuestas atravesaron unos nueve enrutadores.

La dirección de destino es anycast: ese mismo valor se anuncia desde centros de datos en todo el mundo y el enrutamiento entrega el paquete al más cercano. El servidor no está en una máquina concreta a la que se marcó, sino en el nodo de borde más próximo, y eso explica que el tiempo de ida y vuelta sea de 50 milisegundos.

Capa de transporte

La conexión se establece con el saludo de tres vías de TCP, visible en las tramas 79, 80 y 81.


```

79  3.656 s  50308 -> 443  [SYN]          ventana 64800, MSS 1440
80  3.706 s  443 -> 50308  [SYN, ACK]  ventana 65535, MSS 1360
81  3.707 s  50308 -> 443  [ACK]

```

Los 50 milisegundos entre el SYN y el SYN-ACK son el tiempo de ida y vuelta hasta Cloudflare, y establecen el piso para todo lo que va encima: cada par de solicitud y respuesta tardó entre 59 y 73 milisegundos, mientras que el servidor reporta 5 milisegundos de procesamiento. Casi toda la latencia es red. Ambos extremos anuncian tamaños máximos de segmento distintos, 1440 y 1360 bytes, y se usa el menor.

La sesión utilizó dos conexiones. El cliente envía la notificación y encadena de inmediato el tools/list sin esperar, porque una notificación no tiene respuesta que aguardar, y como la primera conexión seguía ocupada se abrió una segunda. Las cinco solicitudes siguientes reutilizaron la primera, amortizando un solo saludo de TCP y uno de TLS en toda la sesión.

Conexión	Paquetes	Bytes	Duración	Transportó
50308	32	15 kB	1.17 s	initialize, notificación y las tres llamadas
50309	14	6.8 kB	0.69 s	tools/list

Capa de aplicación

En esta capa se apilan tres protocolos. El primero es TLS 1.3: el Client Hello de la trama 83 lleva la extensión SNI en claro, con el nombre del servidor, porque es lo que permite a Cloudflare saber qué certificado presentar antes de que exista cifrado. El Server Hello selecciona la suite TLS_AES_256_GCM_SHA384 y, mediante ALPN, el protocolo HTTP/1.1.

The image shows a Wireshark network traffic capture. The top pane displays a list of packets. Packet 83 is a TLSv1.3 Client Hello from 2803:c800:406e:cc18::2606 to 2606:4700:3833::681. Packet 84 is a TLSv1.3 Server Hello from 2606:4700:3833::681 to 2803:c800:406e:cc18::2606. The bottom pane shows the details of packet 83, which is a TLSv1.3 Client Hello. It includes the following fields:

- Frame 83: Packet, 343 bytes on wire (2744 bits), 343 bytes captured (2744 bits) on interface \Device\NPF_{37B984EF-075E-4F8D-B51F-822037B2660}
- Ethernet II, Src: MonMaiPrecis_7a:7b:67 (68:14:01:7a:7b:67), Dst: 52:d6:0d:df:fb:9c (52:d6:0d:df:fb:9c)
- Destination: 52:d6:0d:df:fb:9c (52:d6:0d:df:fb:9c) = IG bit: Locally administered address (this is NOT the factory default)
- Source: MonMaiPrecis_7a:7b:67 (68:14:01:7a:7b:67) = IG bit: Globally unique address (factory default)
- Type: IPv6 (0x86dd)
- Internet Protocol Version 6, Src: 2803:c800:406e:cc18::2606, Dst: 2606:4700:3833::681
- Transmission Control Protocol, Src Port: 50308, Dst Port: 443, Seq: 1361, Ack: 1, Len: 269
- [2 Reassembled TCP Segments (1629 bytes): #0(1360), #0(269)]
- Transport Layer Security

The packet bytes pane shows the raw data of the TLS Client Hello, including the magic number 0x0303, version 3, and the SNI extension.

Figura 3. Client Hello y Server Hello de TLS 1.3.

El segundo es HTTP/1.1. Cada mensaje MCP viaja como un POST hacia la ruta /mcp, y el código de estado se usa solo para el resultado de transporte: 200 cuando hay un mensaje que devolver y 202 para una notificación. El tercero es JSON-RPC 2.0, que viaja en el cuerpo. La siguiente figura muestra el apilamiento completo en una sola trama.

Archivo Edición Visualización Ir Captura Analizar Estadísticas Telefonía Wireless Herramientas Ayuda

http.request or http.response

No.	Time	Source	Destination	Protocol	Length	Info
91	3.827382100	2803:c800:406e:cc18::	2606:4700:3033::681::	HTTP/1.1	636	POST /mcp HTTP/1.1, JSON (application/json)
95	3.900002100	2606:4700:3033::681::	2803:c800:406e:cc18::	HTTP/1.1	101	HTTP/1.1 200 OK, JSON (application/json)
97	4.143287800	2803:c800:406e:cc18::	2606:4700:3033::681::	HTTP/1.1	449	POST /mcp HTTP/1.1, JSON (application/json)
101	4.201893700	2606:4700:3033::681::	2803:c800:406e:cc18::	HTTP	810	HTTP/1.1 202 Accepted
107	4.284406000	2803:c800:406e:cc18::	2606:4700:3033::681::	HTTP/1.1	533	POST /mcp HTTP/1.1, JSON (application/json)
111	4.349345700	2606:4700:3033::681::	2803:c800:406e:cc18::	HTTP/1.1	284	HTTP/1.1 200 OK, JSON (application/json)
113	4.399318300	2803:c800:406e:cc18::	2606:4700:3033::681::	HTTP/1.1	519	POST /mcp HTTP/1.1, JSON (application/json)
115	4.458382500	2606:4700:3033::681::	2803:c800:406e:cc18::	HTTP/1.1	101	HTTP/1.1 200 OK, JSON (application/json)
117	4.489456500	2803:c800:406e:cc18::	2606:4700:3033::681::	HTTP/1.1	580	POST /mcp HTTP/1.1, JSON (application/json)
120	4.554111200	2606:4700:3033::681::	2803:c800:406e:cc18::	HTTP/1.1	101	HTTP/1.1 200 OK, JSON (application/json)
123	4.570737300	2803:c800:406e:cc18::	2606:4700:3033::681::	HTTP/1.1	519	POST /mcp HTTP/1.1, JSON (application/json)
126	4.630624600	2606:4700:3033::681::	2803:c800:406e:cc18::	HTTP/1.1	101	HTTP/1.1 200 OK, JSON (application/json)

> Frame 91: Packet, 636 bytes on wire (5088 bits), 636 bytes captured (5088 bits) on interface \Device\NPF_{37B984EF-075E-4F8D-B51F-82203782660}	0000	52 d6 02 46 e2 fb 9c 68 14	01 7a 7b 67 68 66 dd 60 01	80 00 00 00 00 00 00 00	0000	52 d6 02 46 e2 fb 9c 68 14	01 7a 7b 67 68 66 dd 60 01	80 00 00 00 00 00 00 00
> Ethernet II, Src: HonHaiPrecis_7a:7b:67 (68:14:01:7a:7b:67), Dst: 52:d6:0d:df:fb:9c (52:d6:0d:df:fb:9c)	0000	52 d6 02 46 e2 fb 9c 68 14	01 7a 7b 67 68 66 dd 60 01	80 00 00 00 00 00 00 00	0000	52 d6 02 46 e2 fb 9c 68 14	01 7a 7b 67 68 66 dd 60 01	80 00 00 00 00 00 00 00
> Destination: 52:d6:0d:df:fb:9c (52:d6:0d:df:fb:9c)	0000	52 d6 02 46 e2 fb 9c 68 14	01 7a 7b 67 68 66 dd 60 01	80 00 00 00 00 00 00 00	0000	52 d6 02 46 e2 fb 9c 68 14	01 7a 7b 67 68 66 dd 60 01	80 00 00 00 00 00 00 00
>	0000	52 d6 02 46 e2 fb 9c 68 14	01 7a 7b 67 68 66 dd 60 01	80 00 00 00 00 00 00 00	0000	52 d6 02 46 e2 fb 9c 68 14	01 7a 7b 67 68 66 dd 60 01	80 00 00 00 00 00 00 00
> = LG bit: Locally administered address (this is NOT the factory default)	0000	52 d6 02 46 e2 fb 9c 68 14	01 7a 7b 67 68 66 dd 60 01	80 00 00 00 00 00 00 00	0000	52 d6 02 46 e2 fb 9c 68 14	01 7a 7b 67 68 66 dd 60 01	80 00 00 00 00 00 00 00
> = IG bit: Individual address (unicast)	0000	52 d6 02 46 e2 fb 9c 68 14	01 7a 7b 67 68 66 dd 60 01	80 00 00 00 00 00 00 00	0000	52 d6 02 46 e2 fb 9c 68 14	01 7a 7b 67 68 66 dd 60 01	80 00 00 00 00 00 00 00
> Source: HonHaiPrecis_7a:7b:67 (68:14:01:7a:7b:67)	0000	52 d6 02 46 e2 fb 9c 68 14	01 7a 7b 67 68 66 dd 60 01	80 00 00 00 00 00 00 00	0000	52 d6 02 46 e2 fb 9c 68 14	01 7a 7b 67 68 66 dd 60 01	80 00 00 00 00 00 00 00
> = LG bit: Globally unique address (factory default)	0000	52 d6 02 46 e2 fb 9c 68 14	01 7a 7b 67 68 66 dd 60 01	80 00 00 00 00 00 00 00	0000	52 d6 02 46 e2 fb 9c 68 14	01 7a 7b 67 68 66 dd 60 01	80 00 00 00 00 00 00 00
> = IG bit: Individual address (unicast)	0000	52 d6 02 46 e2 fb 9c 68 14	01 7a 7b 67 68 66 dd 60 01	80 00 00 00 00 00 00 00	0000	52 d6 02 46 e2 fb 9c 68 14	01 7a 7b 67 68 66 dd 60 01	80 00 00 00 00 00 00 00
> Type: IPv6 (0x86dd)	0000	52 d6 02 46 e2 fb 9c 68 14	01 7a 7b 67 68 66 dd 60 01	80 00 00 00 00 00 00 00	0000	52 d6 02 46 e2 fb 9c 68 14	01 7a 7b 67 68 66 dd 60 01	80 00 00 00 00 00 00 00
> [Stream index: 0]	0000	52 d6 02 46 e2 fb 9c 68 14	01 7a 7b 67 68 66 dd 60 01	80 00 00 00 00 00 00 00	0000	52 d6 02 46 e2 fb 9c 68 14	01 7a 7b 67 68 66 dd 60 01	80 00 00 00 00 00 00 00
> Internet Protocol Version 6, Src: 2803:c800:406e:cc18:5d52:3e3b:eb6:fbc6, Dst: 2606:4700:3033::6815:3f1e	0000	52 d6 02 46 e2 fb 9c 68 14	01 7a 7b 67 68 66 dd 60 01	80 00 00 00 00 00 00 00	0000	52 d6 02 46 e2 fb 9c 68 14	01 7a 7b 67 68 66 dd 60 01	80 00 00 00 00 00 00 00
> Transmission Control Protocol, Src Port: 50308, Dst Port: 443, Seq: 1630, Ack: 3924, Len: 562	0000	52 d6 02 46 e2 fb 9c 68 14	01 7a 7b 67 68 66 dd 60 01	80 00 00 00 00 00 00 00	0000	52 d6 02 46 e2 fb 9c 68 14	01 7a 7b 67 68 66 dd 60 01	80 00 00 00 00 00 00 00
> Transport Layer Security	0000	52 d6 02 46 e2 fb 9c 68 14	01 7a 7b 67 68 66 dd 60 01	80 00 00 00 00 00 00 00	0000	52 d6 02 46 e2 fb 9c 68 14	01 7a 7b 67 68 66 dd 60 01	80 00 00 00 00 00 00 00
> Hypertext Transfer Protocol	0000	52 d6 02 46 e2 fb 9c 68 14	01 7a 7b 67 68 66 dd 60 01	80 00 00 00 00 00 00 00	0000	52 d6 02 46 e2 fb 9c 68 14	01 7a 7b 67 68 66 dd 60 01	80 00 00 00 00 00 00 00
> JavaScript Object Notation: application/json	0000	52 d6 02 46 e2 fb 9c 68 14	01 7a 7b 67 68 66 dd 60 01	80 00 00 00 00 00 00 00	0000	52 d6 02 46 e2 fb 9c 68 14	01 7a 7b 67 68 66 dd 60 01	80 00 00 00 00 00 00 00

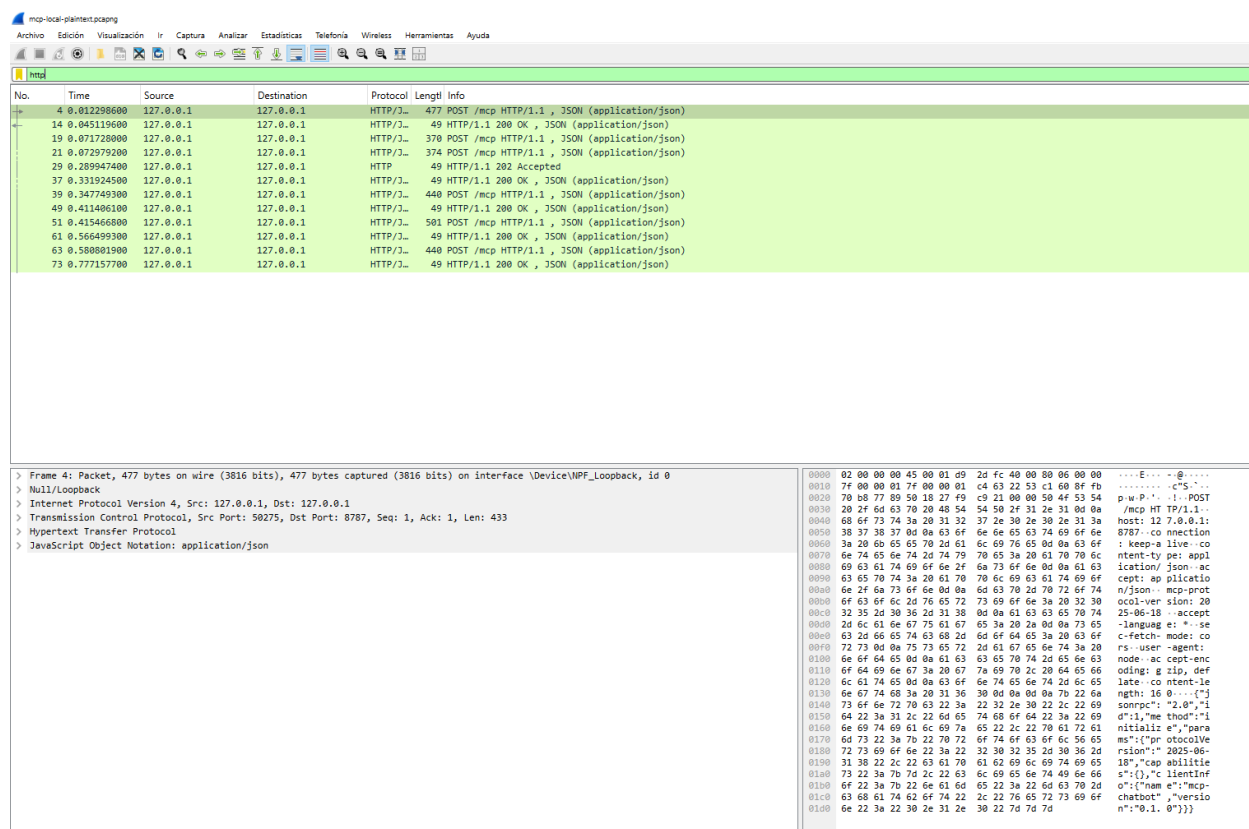


Figura 5. La misma sesión sobre loopback sin cifrado. El mensaje initialize se lee en texto plano.

Capa	Sesión remota	Sesión local
Enlace	Ethernet II sobre Wi-Fi, MTU de 1500	Loopback, sin medio físico
Red	IPv6 a dirección anycast, límite de saltos 63	IPv4 de 127.0.0.1 a 127.0.0.1, TTL 128
Transporte	TCP puerto 443, MSS 1440 y 1360	TCP puerto 8787, MSS 65495
Seguridad	TLS 1.3, se requieren claves para leer	Ninguna, contenido legible directamente
Tiempo de ida y vuelta	Alrededor de 50 milisegundos	Menos de 1 milisegundo

La diferencia más reveladora es el tamaño máximo de segmento: 65495 bytes en loopback contra 1440 sobre Wi-Fi. Loopback nunca toca una tarjeta de red y no está limitado por la MTU de Ethernet, así que un mensaje completo cabe en un solo segmento.

Aun así, en la capa JSON-RPC las dos capturas son idénticas: las mismas solicitudes, la misma notificación y las mismas respuestas asociadas por los mismos identificadores. A MCP no le consta que una sesión cruzó el internet y la otra nunca salió de la máquina, que es el propósito de un modelo por capas.

4. Dificultades encontradas

- El primero fue que la caché de npx estaba corrupta y el servidor Filesystem oficial no arrancaba. Se resolvió instalándolo como dependencia del proyecto. El servidor Git tuvo un problema equivalente con uvx y se instaló con pip.
- El segundo fue que el servidor Git oficial no expone ninguna herramienta para crear repositorios en ninguna de sus versiones publicadas. Por eso el repositorio de demostración se crea durante la instalación, y el chatbot se encarga de escribir el archivo, agregarlo y hacer el commit.
- El tercero fue que un servidor que no lograba arrancar dejaba al chatbot esperando los 30 segundos completos del tiempo de espera. Detectar la salida del proceso hijo convirtió esa espera en un error inmediato.
- El cuarto fue el más instructivo. El código armaba el mensaje del asistente a partir de un formato interno propio, y eso descartaba campos que solo el proveedor conoce, como reasoning en Groq y thoughtSignature en Gemini. El modelo se detenía a medio camino o la petición era rechazada. Se resolvió guardando el mensaje tal como llegó y reenviándolo sin modificarlo.
- El quinto fueron los límites de las capas gratuitas. El catálogo de 30 herramientas viaja en cada petición y consume unos 2300 tokens, lo que agotaba la cuota de Groq. Se recortaron las descripciones y se agregaron reintentos automáticos.
- El sexto fue que Gemini valida los esquemas de las herramientas de forma estricta y rechaza la petición completa por una palabra clave que no reconoce. Los esquemas se reescriben al subconjunto que acepta antes de enviarlos.

5. Lecciones aprendidas

- El resultado más claro es uno negativo: mover el servidor de logística desde una tubería local hasta un centro de datos de Cloudflare no requirió ningún cambio en el cliente MCP. El enlace inicial, la asociación por identificadores y las llamadas son el mismo código en ambos casos, solo se agregó un transporte.. Ese es el argumento a favor de un protocolo estándar.
- Implementar el protocolo a mano obligó a tomar decisiones que un SDK habría resuelto en silencio: que la delimitación sobre stdio es por saltos de línea, que una notificación no lleva identificador y por eso el transporte HTTP devuelve un cuerpo vacío, y que las respuestas se asocian por identificador y no por orden de llegada.
- El modelo por capas dejó de ser abstracto al comparar las dos capturas. Los mismos mensajes aparecen en ambas, aunque por debajo una usó IPv6 hacia una dirección anycast con TLS encima

y la otra nunca salió de la máquina. También quedó claro que la red domina la experiencia: cinco milisegundos de procesamiento contra cincuenta de ida y vuelta.

6. Conclusiones

- El proyecto cumplió con las diez funcionalidades requeridas. El chatbot responde con su propia base de conocimiento, mantiene el contexto de la sesión, registra cada mensaje JSON-RPC en ambas direcciones y maneja cuatro servidores MCP: dos oficiales, uno propio y ese mismo servidor propio otra vez a través de la red.
- El valor de MCP no está en su complejidad técnica sino en que todos lo cumplen. Es un acuerdo delgado sobre JSON-RPC 2.0: un enlace inicial, una forma de listar herramientas y una forma de invocarlas. Los servidores oficiales se escribieron sin conocer este chatbot y el servidor de logística se escribió sin conocer ningún anfitrión, y aun así interoperan.
- El initialize deja de ser una llamada a una función y pasa a ser un saludo de TCP, una negociación de TLS, una petición HTTP y un objeto JSON, cada uno visible en Wireshark como una capa distinta haciendo su propio trabajo.